GRAFO

Guia do desenvolvedor

Como o Grafo é organizado, onde fica cada coisa, como testar e publicar, e o que cada recurso faz por dentro.

Versão de 20 de outubro de 2026 · Repositório `iago069/Grafo-project` · Produção em `thegrafo.vercel.app`
Este guia substitui a versão anterior e inclui: biblioteca, equipes com vários diagramas e acesso por diagrama, painel do administrador, leitor de texto e de imagens com IA, ferramentas de desenho, ligação de cabos com o mouse, links entre diagramas, camadas e comparação com o NetBox.

Sumário

1. Visão geral e regras do projeto
2. Como rodar, testar e publicar
3. Estrutura de pastas
4. O editor (`src/app`) e como ele é montado
5. Formato dos dados de um diagrama
6. Conta, nuvem e sem internet
7. Banco de dados (Supabase) e migrações
8. Equipes, grupos e acesso por diagrama
9. Administrador, temas e bloqueio
10. Início e Biblioteca
11. Ferramentas de desenho e cabos
12. Leitor de texto e de imagens (IA)
13. Outros módulos
14. Testes
15. Segurança
16. Como acrescentar um recurso novo
17. Pendências conhecidas

1. Visão geral e regras do projeto

O Grafo desenha topologia de rede (equipamentos, portas e cabos), monta racks (Bayface), guarda documentação (Biblioteca) e permite trabalhar em equipe. É uma aplicação web em **JavaScript puro**, sem framework e sem etapa de build no navegador: os arquivos de `assets/` são servidos como estão.

- **Desenho:** o Graphviz (Viz.js, via CDN) calcula o layout inicial e gera o SVG. Depois disso o próprio Grafo controla posições, portas e a rota dos cabos.
- **Dados:** cada diagrama é um objeto JSON validado por `assets/js/comum/esquema.js` (Zod).
- **Nuvem:** Supabase (Auth, Postgres com RLS, Storage, Realtime).
- **Hospedagem:** Vercel. Um `git push` na branch `main` publica sozinho. Funções de servidor ficam em `api/`.

> **Regras que valem sempre**
> Textos da interface e comentários em português, frases curtas. Nunca usar a chave `service_role` /secreta do Supabase no código nem no navegador. `.env` não vai para o Git. Só publicar com as duas baterias de testes passando. Ninguém vira administrador pelo app: só pela tabela `grafo_admins`, editada no Supabase.

2. Como rodar, testar e publicar

```
npm install                # dependências (express, @anthropic-ai/sdk, playwright)
npm start                  # servidor local em http://localhost:3000 (scripts/servidor-local.js)
npm run montar             # junta src/app/*.js em assets/js/editor/app.js
npm test                   # testes de unidade (node --test, tests/unidade)
npm run test:navegador     # testes no Chromium (Playwright, tests/navegador)
```

Publicar

1. Edite em `src/app/` (editor) ou em `assets/` (demais módulos).
2. Rode `npm run montar` se mexeu em `src/app/`. O teste `montagem.test.js` falha se o `app.js` estiver desatualizado.
3. Rode `npm test` e `npm run test:navegador`. Só siga se os dois mostrarem `fail 0`.
4. `git commit` e `git push` na `main`. A Vercel roda `npm test` no build (`vercel.json → buildCommand`) e publica.
5. Confira o status do deploy: `gh api repos/iago069/Grafo-project/commits/<hash>/statuses` (contexto da Vercel).

Variáveis de ambiente (Vercel → Settings → Environment Variables)

Variável	Usada por	Observação
`SUPABASE_URL`	`api/auth-config.js`, `api/ler-imagem.js`	Endereço do projeto Supabase.
`SUPABASE_PUBLISHABLE_KEY`	idem	Chave pública (anon). Pode ir ao navegador.
`ANTHROPIC_API_KEY`	`api/ler-imagem.js`	Leitura de prints com IA. **Nunca** vai ao navegador. Sem ela, a função responde 503.

`vercel.json` também define o cabeçalho de segurança (CSP), as reescritas (`/ver` → `ver.html`, o resto → `index.html`) e a pasta de saída (a raiz).

3. Estrutura de pastas

Caminho	O que tem
`index.html`	Página única: landing, login, editor, todas as janelas.
`ver.html`	Visualização somente leitura de um link compartilhado.
`src/app/NN-*.js`	Fatias do editor, juntadas em ordem alfabética em `assets/js/editor/app.js`.
`assets/js/conta/`	`auth.js` (login, gravação na nuvem, sem internet, carrega os módulos) e `sessao.js`.
`assets/js/comum/`	`esquema.js` (validação), `interface.js` (menus, temas), `erros.js` (registro de erros).
`assets/js/recursos/`	Módulos independentes do editor (biblioteca, equipe, admin, leitor, racks…), carregados depois do login.
`assets/css/styles.css`	Todo o estilo. Blocos novos vão no fim, com um comentário de título.
`assets/images/equipamentos/`	Ícones dos tipos de equipamento.
`data/equipamentos.json`	Catálogo de tipos e modelos (portas por modelo).
`api/`	Funções da Vercel: `auth-config`, `equipamentos`, `ler-imagem`.
`supabase/migracoes/`	SQL para rodar no SQL Editor do Supabase, em ordem de data.
`agente/`	Agente opcional para NetBox e status online (fica oculto na interface).
`tests/unidade`, `tests/navegador`	Testes. `tests/unidade/simulador.js` e `tests/navegador/ajuda.js` simulam o Supabase.
`sw.js`	Service worker: guarda os arquivos para abrir sem internet.

4. O editor (`src/app`)

O editor é **um único bloco** `(function () { … })()` com estado compartilhado. As partes são fatias desse bloco: o `00` abre o bloco e o `12` fecha. Funções declaradas em uma parte são vistas em todas. `npm run montar` concatena tudo.

Parte	Responsabilidade
`00-catalogo`	Tipos de equipamento, ícones (`iconePorTipo`), modelos e portas sugeridas.
`01-estado-e-vista`	`estado` do diagrama aberto, referências de elementos, ids.
`02-posicoes-e-portas`	Posição absoluta de cada equipamento (`eq.posicao` = centro) e lado das portas.
`03-cabos-grupos-e-arraste`	Rota ortogonal dos cabos (`rotearCabosOrtogonalmente`), caixas dos grupos, arrastar, seleção, copiar/colar, agrupar, anotações.
`04-busca-caminho-e-limpar`	Busca, rastrear caminho, limpar.
`05-zoom-modais-e-editor`	Zoom e pan, toque, janelas (`abrirModal`), editores de equipamento e de cabo.

`06-projetos-historico-e-backup`	Lista de projetos, desfazer/refazer (`salvarLocal` registra o passo), backup, importar JSON.
`07-temas`	Temas (incluindo o exclusivo "Fernando").
`08-listas-e-formularios`	Formulários da barra lateral, `renderTudo()`, limites.
`09-csv-e-mensagens`	CSV, "Aplicar JSON", avisos (`avisar`, com "Desfazer").
`10-desenho-e-ferramentas`	Caixa de ferramentas, formas e caneta, etiqueta de VLAN, rota à mão, biblioteca de equipamentos, ligar com o mouse, links entre diagramas, camadas.
`10-diagrama-e-layout`	`gerar()`: renderiza, posiciona, roteia e liga tudo; `window.GrafoApp`; Organizar.
`11-exportacao-svg-png`	SVG e PNG (inclusive com papel escuro).
`12-pagina-e-pdf`	PDF para imprimir e inicialização final.

Ciclo de renderização

`renderTudo()` atualiza listas, salva e agenda `gerar()`. O `gerar()` faz, nesta ordem: Graphviz → `sincronizarPosicoes` → posições salvas → caixas dos grupos → `rotearCabosOrtogonalmente` (com etiquetas de VLAN e números) → `desenharDesenhos` → `desenharAnotacoes` → viewBox → ligações de eventos → imagens → selos de link → `aplicarCamadas`. No fim dispara o evento `grafo:render`, que outros módulos escutam.

> **Mudanças leves**
> Mexer só em desenhos, etiquetas ou rota de cabo não precisa do Graphviz: `confirmarMudanca()` redesenha só essas camadas, chama `salvarLocal()` (que cria o passo do Ctrl+Z) e dispara `grafo:render`.

API para os módulos: `window.GrafoApp`

`projetoAtual`, `nomesProjetos`, `dadosProjeto`, `substituirProjeto`, `trocarProjeto`, `removerProjeto`, `editarEquipamento`, `janela` (janela do Grafo no lugar de `prompt` / `confirm`), `avisar`, `salvar`, `redesenhar`, `svgDoDiagrama`, entre outros. Módulos novos devem usar só essa API.

5. Formato dos dados de um diagrama

```
{
  "equipamentos": [{ "id", "nome", "tipo", "modelo", "ip", "grupo", "status", "icone", "imagem",
                     "corFonte", "corIcone"?, "rede"?, "diagrama"?, "posicao": {x,y}, "deslocamento":
{x,y} }],
  "conexoes":     [{ "de", "para", "saida", "chegada", "tipo", "potencia", "cor", "grossura", "estilo",
                     "vlan"?, "redeVlan"?, "numSaida"?, "numChegada"?, "etiqueta"?: {x,y}, "rota"?:
[{x,y}…] }],
  "anotacoes":    [{ "id", "texto", "x", "y", "cor" }],
  "desenhos":     [{ "id", "tipo": texto|retangulo|elipse|seta|linha|caneta, "x","y","x2"?,"y2"?,
"pontos"?,
                     "cor", "fundo"?, "espessura", "tracejado"?, "texto"?, "tamanho"? }],
  "racks", "cabosRack", "documentos", "modo"   // opcionais
}
```

- **Validação:** `Esquema.validarProjeto(dados, "estrito")` na importação (recusa erros) e `"reparar"` ao carregar o que já estava salvo (corrige e avisa). Os limites ficam em `LIMITES`: 35 equipamentos por diagrama, 300 desenhos, 40 dobras por rota, 40 caracteres no tipo do cabo, entre outros.
- `diagrama`**:** nome de outro diagrama aberto pelo selo ↗ do equipamento.
- `rota`**:** guarda a rota inteira, com as duas pontas. Se a porta de um dos lados se mexer mais de 30 px (arrastar ou Organizar), a rota é ignorada e o cabo volta para a rota automática. Isso evita voltas pelo desenho.
- **Organizar:** apaga posições, rotas e deslocamentos das etiquetas, e traz para perto as notas e desenhos que estavam longe.

6. Conta, nuvem e sem internet

- `auth.js` busca a configuração em `/api/auth-config`, cria o cliente Supabase e carrega o editor e os módulos (`loadApp`). A lista `keepEditorOffline` diz quais arquivos o service worker guarda.
- **Diagramas pessoais:** numa linha por conta em `grafo_user_data`, com proteção contra gravação por cima: a gravação compara o `updated_at` e, se outra aba salvou antes, abre o aviso de conflito.
- **Diagramas de equipe:** cada um numa linha de `grafo_projetos_equipe`, com a mesma proteção (`equipe.js → gravar`). `separar()` tira esses diagramas da gravação pessoal.
- **Sem internet:** as alterações ficam no navegador e sobem quando a conexão volta.
- **Sem login:** projetos ficam só no navegador. Compartilhar, Histórico e Equipe pedem login.

7. Banco de dados (Supabase) e migrações

Cada arquivo de `supabase/migracoes/` é rodado **uma vez**, à mão, no SQL Editor, em ordem de data. Todos terminam com `notify pgrst, 'reload schema'`, para a API ver as funções novas na hora. Todas as tabelas usam RLS. Funções que precisam passar por cima das regras são `security definer`, com `search_path = ''`.

Migração	Cria ou muda
`2026-09-28-compartilhar`	`grafo_compartilhamentos`: links só para ver.
`2026-09-28-excluir-conta`	Função `excluir_minha_conta`.
`2026-09-29-equipes`	`grafo_projetos_equipe`, `grafo_membros`, `grafo_convites`, Realtime.
`2026-09-29-historico`	`grafo_versoes`: histórico de versões.
`2026-09-30-erros`	`grafo_erros`: erros do navegador.
`2026-10-15-admin`	`grafo_admins`, `grafo_eh_admin`, painel. O e-mail fica como `SEU_EMAIL_AQUI` no Git.
`2026-10-15-documentos`	Regras do bucket `documentos` no Storage.
`2026-10-15-temas-liberados`	`grafo_temas_liberados`: tema exclusivo por pessoa.
`2026-10-16-admin-permissoes`	`grafo_bloqueados`, `grafo_avisos`, `grafo_minha_situacao`, funções do admin, regras restritivas.
`2026-10-16-biblioteca`	`grafo_biblioteca`: estantes, livros e páginas.
`2026-10-19-equipe-grupos`	`grafo_grupos`, `grafo_convites_grupo`, `papel` em `grafo_membros`, `grupo` / `papel_padrao` nos diagramas, regra "só edita quem tem papel editar".
`2026-10-19-equipe-agrupar`	`grafo_agrupar`: transforma equipes antigas (um diagrama) em grupo.

8. Equipes, grupos e acesso por diagrama

`assets/js/recursos/equipe.js`. Uma **equipe** (grupo) junta vários diagramas do mesmo dono, com **um link de convite**. Cada diagrama continua sendo uma linha própria em `grafo_projetos_equipe`, o que mantém a gravação, o conflito e o tempo real iguais aos de antes.

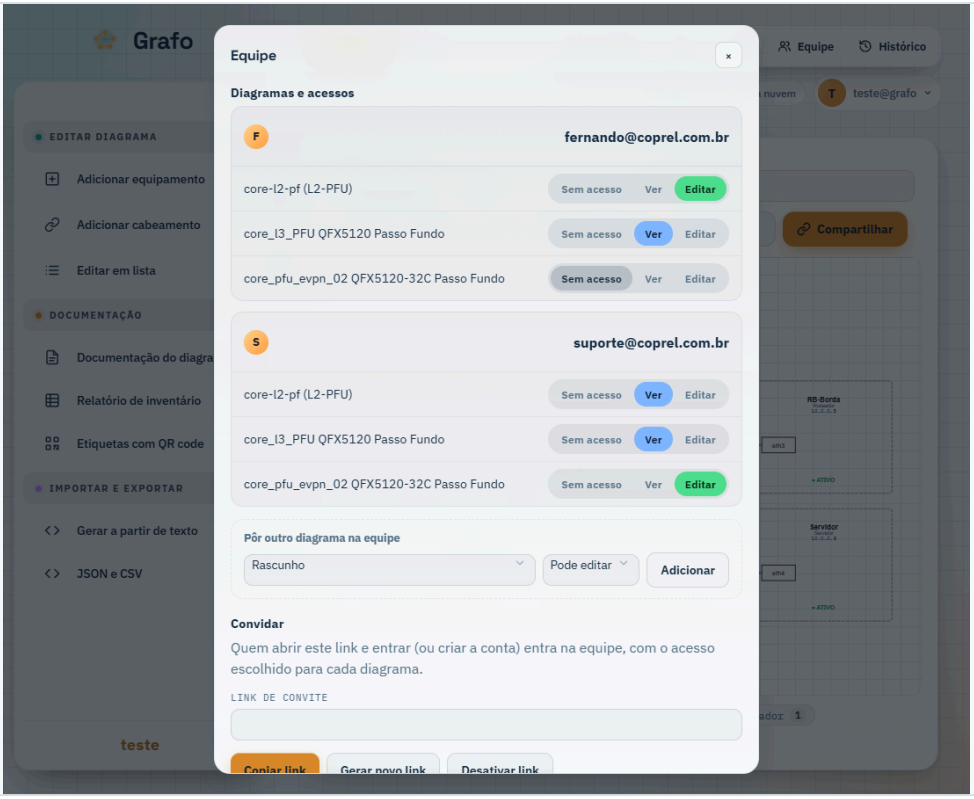
Conceito	Onde fica
Grupo e dono	`grafo_grupos`; o diagrama aponta pela coluna `grupo`.
Acesso de cada pessoa a cada diagrama	Uma linha em `grafo_membros` com `papel` `ver` ou `editar`. Sem linha = sem acesso.
Acesso de quem entrar pelo convite	`papel_padrao` do diagrama.
Convite	`grafo_convites_grupo` (grupo) ou `grafo_convites` (diagrama antigo). O link é sempre `/?convite=<token>`: o app tenta primeiro o de grupo.

Funções (RPC)

`grafo_criar_grupo`, `grafo_adicionar_ao_grupo`, `grafo_aceitar_convite_grupo`, `grafo_definir_acesso` (só o dono; `nenhum` apaga a linha), `grafo_agrupar` (equipes antigas). A regra de UPDATE em `grafo_projetos_equipe` exige `grafo_pode_editar`, então quem só vê não grava, mesmo mexendo no navegador.

No navegador

- Quem só vê: `gravar()` pula o diagrama, o `body` ganha `data-somente-leitura` e qualquer alteração é desfeita ao ouvir `grafo:render`.
- Se o dono troca alguém para "só ver" com o diagrama aberto, a gravação seguinte falha, o app confere o papel, volta para a versão do servidor e avisa (sem janela de conflito).
- API exposta: `GrafoEquipe.criarGrupo`, `adicionarAoGrupo`, `acessosDoGrupo`, `definirAcesso`, `agrupar`, `papel`, `grupo`.



Janela Equipe do dono: um cartão por pessoa e três botões por diagrama.

9. Administrador, temas e bloqueio

`assets/js/recursos/admin.js`. O item "Administração" aparece no menu da conta só se `grafo_eh_admin()` responder verdadeiro. Ninguém vira admin pelo app: a conta precisa estar em `grafo_admins`, que só se edita no Supabase.

- Painel: liga e desliga admin, bloqueio e tema exclusivo por pessoa (`data-campo` = `admin`, `bloqueio`, `tema:fernando`), com o botão "Salvar alterações". Também publica um aviso para todos.
- Cada conta confere a própria situação (`grafo_minha_situacao`) a cada 60 s e ao voltar para a aba: conta bloqueada vê a tela de bloqueio; o aviso aparece numa faixa.
- O tema "Fernando" fica escondido (`data-tema-exclusivo`) e só aparece para quem tem liberação em `grafo_temas_liberados`.

10. Início e Biblioteca

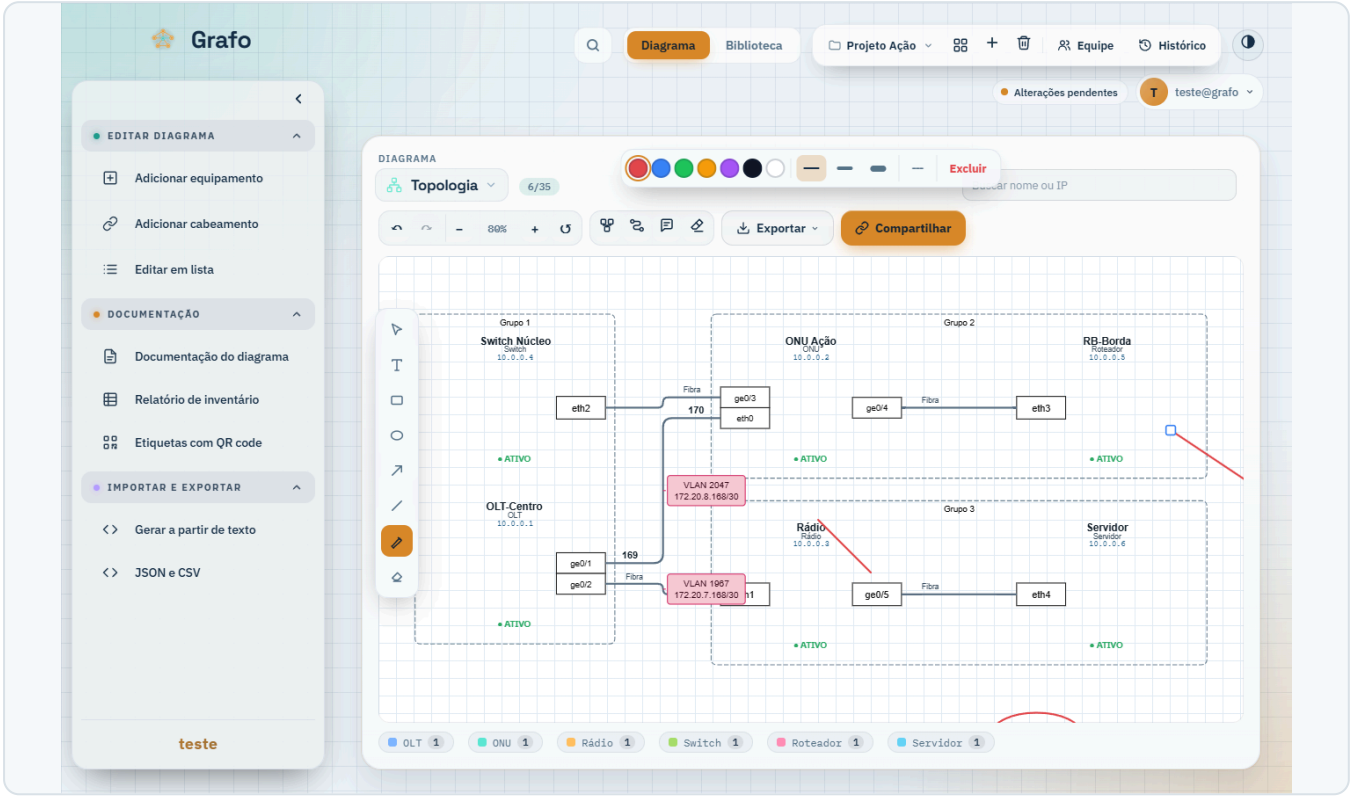
`assets/js/recursos/biblioteca.js`. O app abre onde a pessoa parou (diagrama ou Biblioteca), lembrado em `grafo-ultima-vista`. O topo tem só "Diagrama | Biblioteca"; a tela de início animada abre pela logo.

- Biblioteca no estilo BookStack: estantes → livros → capítulos → páginas (markdown simples), guardada em `grafo_biblioteca` e no navegador. Anexos vão para o bucket `documentos`.
- "Documentação do diagrama" abre o livro do diagrama na estante POPs (criado na hora, se não existir).
- Livros aparecem de lado na estante e são "puxados" ao clicar.

11. Ferramentas de desenho e cabos

Tudo em `src/app/10-desenho-e-ferramentas.js`. A caixa de ferramentas fica à esquerda do quadro.

Recurso	Como funciona por dentro
Biblioteca de equipamentos	Lista `catalogoTiposEquipamentos`. Clique → `adicionarEquipamentoEm` no centro visível; arrastar usa drag-and-drop HTML com o tipo `application/x-grafo-equip` e grava `posicao` onde soltou.
Ligar com o mouse	Ao passar o mouse num nó aparecem 4 bolinhas (`g.ligar-alcas`). Arrastar até outro nó cria a conexão validada e abre `abrirEditorCabo`.
Formas, texto, caneta, borracha	Ficam em `estado.desenhos`, desenhadas em `g.camada-desenhos` dentro do SVG (saem na exportação). Arrastes usam ouvintes na `window`, para o elemento poder ser recriado sem soltar o mouse. Durante o arraste só o desenho mexido é redesenhado (`redesenharUm`).
Etiqueta de VLAN e números	`desenharExtrasDoCabo`, chamada a cada rota calculada. A etiqueta fica presa ao meio do cabo e guarda o deslocamento em `conexao.etiqueta`.
Rota à mão	Clique seleciona o cabo e desenha alças no meio de cada trecho. Arrastar uma alça move o trecho e redesenha só aquele cabo; ao soltar, grava `conexao.rota`.
Link entre diagramas	Selo ↗ (`g.selo-diagrama`) nos equipamentos com `diagrama`. Abre o outro diagrama e mostra "← Voltar para …".
Camadas	Esconde por seletor (VLAN, números, nomes dos cabos, desenhos, notas, grupos) com estilo inline, então vale também na exportação. A escolha fica em `localStorage` (`grafo-camadas:<diagrama>`).
Como fazer	Ajuda com busca, em `comandos.js` (`window.GrafoAjuda`), aberta pelo botão "?".



Caixa de ferramentas, etiquetas de VLAN, números nas pontas e formas desenhadas.

12. Leitor de texto e de imagens (IA)

`assets/js/recursos/leitor.js` ("Gerar a partir de texto"). Sem IA, reconhece: tabela de interfaces do **Grafana**, do **NetBox**, `show interfaces descriptions` e `show lldp neighbors` do Junos.

1. Cada leitor gera registros por porta (`porta` , `velocidade` , `desc` , `lag` , `vizinho` , `portaVizinho`).
2. `juntar()` combina por porta com prioridade: Grafana e Junos (3) > imagem lida com IA (2) > NetBox e LLDP (1). O vizinho vem do LLDP, depois do NetBox.
3. Porta com descrição "aeN" entra na LAG aeN; o destino vem da descrição da LAG. Cada porta física vira um cabo. Porta que só existe no NetBox entra tracejada. Sem destino vira nota.
4. A revisão permite desmarcar cabos e depois **criar um diagrama** (reaproveita a importação de JSON) ou **mesclar no aberto** (`GrafoLeitor.mesclar` , aplicado pelo "Aplicar JSON", com Ctrl+Z).
5. `GrafoLeitor.comparar` confronta o texto com o diagrama aberto: iguais, diferentes, faltam e só no diagrama.

Imagens com IA: `api/ler-imagem.js`

- O navegador reduz a imagem (máx. 1800 px, JPEG) e envia com o token da sessão do Supabase. A função confere o token em `/auth/v1/user` e recusa quem não está logado.
- Chama o Claude (`claude-opus-5-5` , SDK `@anthropic-ai/sdk`) com saída em JSON Schema (portas e notas). O resultado entra no mesmo leitor, como fonte `ia` .
- Recusa por segurança: a chamada usa a opção do servidor que tenta outro modelo (`fallbacks: "default"`); se a API não aceitar o parâmetro, repete sem ele. `stop_reason` `refusal` e `max_tokens` viram mensagens claras.

13. Outros módulos (`assets/js/recursos`)

Arquivo	O que faz
`comandos.js`	Paleta Ctrl+K, cartão ao passar o mouse, folha de atalhos (?), galeria de projetos, Como fazer.
`racks.js`	Bayface: racks, U, fotos dos modelos, visão de datacenter por fileiras.
`compartilhar.js`	Links só para ver (`grafo_compartilhamentos` , abertos em `ver.html`).
`historico.js`	Versões do diagrama (`grafo_versoes`).
`etiquetas.js`	PDF A4 de etiquetas com QR code.
`planilha.js`	Relatório de inventário em .xlsx, gerado no navegador.
`primeiros-passos.js`	Cartão das três primeiras tarefas.
`integracao.js`	NetBox e status online via agente local (oculto por enquanto).
`instalar.js`	Botões "Instalar app" (PWA).

14. Testes

Unidade: 88 testes

`npm test` . Sem navegador. `simulador.js` imita o Supabase (com as regras das tabelas de equipe, grupos, papéis e convites) e um editor falso, para testar `auth.js` e `equipe.js` com várias contas.

Navegador: 72 testes

`npm run test:navegador` . Chromium via Playwright, com o Supabase simulado por `tests/navegador/ajuda.js` (`abrirPagina` , `esperarDiagrama` ; respostas de RPC em `window.__respostasRpc`). Os testes abrem direto no diagrama e com todos os menus abertos.

> **Regra**
> Todo recurso novo ganha teste. Todo teste confere `pagina.erros` vazio (nenhum erro no console). Funções que chamam serviços externos (IA) são testadas com a rota simulada (`pagina.route`).

15. Segurança

- **Chaves:** só a chave pública do Supabase vai ao navegador. A chave `service_role` nunca é usada no projeto. A chave da Anthropic fica só na Vercel.
- **Acesso aos dados:** o que cada conta vê e grava é decidido por RLS no banco, não pelo navegador. Esconder um botão nunca é a única proteção.
- **Admin:** só pela tabela `grafo_admins`, sem caminho no app que escreva nela.
- **Conteúdo:** textos vindos de dados são escapados antes de ir para o HTML (`escaparHtml` / `esc`). O CSP do `vercel.json` limita scripts ao próprio site e ao jsDelivr.
- **Leitura com IA:** exige login e limita o tamanho da imagem (≈ 4,5 MB) e da resposta (400 portas).

16. Como acrescentar um recurso novo

1. **Dentro do editor?** Crie ou edite uma parte em `src/app` e rode `npm run montar`. **Separado?** Crie um módulo em `assets/js/recursos` que use `window.GrafoApp`, carregue em `auth.js → loadApp` e inclua na lista de `keepEditorOffline`.
2. **Campo novo nos dados?** Acrescente em `esquema.js` (como `opcional`, para não quebrar projetos antigos) e no limite correspondente.
3. **Tabela ou função nova no banco?** Crie `supabase/migracoes/AAAA-MM-DD-nome.sql` com RLS e `notify pgrst`; trate no app o erro "função não encontrada" (SQL ainda não rodado) com uma mensagem clara.
4. **Visual:** estilo no fim de `styles.css`, usando as variáveis do tema (`--text`, `--line-soft`, `--accent`…). Confira no celular: o teste "celular" falha se algo passar da largura da tela.
5. **Teste e publique:** escreva os testes, rode as duas baterias e publique.

17. Pendências conhecidas

- Leitura com IA: falta configurar `ANTHROPIC_API_KEY` na Vercel.
- O teste "abre e edita sem internet" às vezes falha na bateria completa e passa sozinho.
- NetBox pelo agente local está oculto na interface (o código continua).
- Rotas feitas à mão não acompanham o equipamento quando ele é arrastado: voltam para a automática (de propósito).